

Digital Certificates

1. Overview

A **digital certificate** (also called a public key certificate or identity certificate) is an **electronic document that binds a public key to an identity**, digitally signed by a trusted Certificate Authority (see the PKI guide) to vouch for that binding.

Certificates are the concrete artifact that makes PKI's trust model usable in practice — the thing a system actually presents, transmits, and verifies.

Digital Certificates at a Glance

Property	Detail
Purpose	Prove ownership of a public key, tied to a verified identity
Format standard	X.509 (the dominant standard in modern use)
Signed by	A Certificate Authority, using the CA's own private key
Contains	Subject identity, public key, issuer, validity period, and more

2. What a Certificate Contains

A digital certificate contains several fields that establish the identity-to-public-key binding and provide information needed for validation.

Field	Meaning
Subject	The name of the entity the certificate identifies — a person, organization, or, most commonly for web certificates, a domain name.
Subject's Public Key	The public key being bound to the subject's identity.
Issuer	The Certificate Authority's name — who signed and vouches for this certificate.
Serial Number	A unique identifier assigned by the issuing CA, used in revocation lists and lookups.
Validity Period	Not Before and Not After dates defining the certificate's active lifetime.
Signature Algorithm	The algorithm used by the CA to sign the certificate, such as SHA-256 with RSA or ECDSA.

CA's Digital Signature	The signature itself — computed over the certificate's other fields using the CA's private key (see the Digital Signatures guide).
Extensions	Additional fields such as Subject Alternative Names (SANs — allowing one certificate to cover multiple domains), key-usage restrictions, and CA constraints.

3. Certificate Validation Levels

Not all certificates involve the same depth of identity verification before issuance. This affects what a relying party can actually infer from a valid certificate.

Validation Level	What's Verified	Typical Use
Domain Validation (DV)	Only that the requester controls the domain in question, for example via a DNS record or a file placed on the web server.	The vast majority of modern website certificates; fast and often automated, such as Let's Encrypt.
Organization Validation (OV)	Domain control, plus verification that the requesting organization is a legitimate registered entity.	Business websites wanting a stronger identity assertion.
Extended Validation (EV)	The most rigorous vetting — legal, physical, and operational existence of the organization is verified.	Historically used for high-visibility organizational trust signals; browser UI treatment of EV has diminished significantly in recent years, reducing its practical differentiation from DV/OV in most users' experience.

Important Nuance

Even the highest validation level only confirms **identity/domain control at issuance time** — it says nothing about whether the site is trustworthy, safe, or free of malicious content.

A certificate proves **"this is genuinely example.com,"** not **"example.com is safe."**

4. Certificate Types by Scope

Type	Coverage
Single-domain	Covers exactly one fully qualified domain name.
Wildcard	Covers a domain and all its direct subdomains, for example <code>*.example.com</code> .
Multi-domain (SAN)	Covers multiple explicitly listed, potentially unrelated domains via the Subject Alternative Name (SAN) extension.

5. The Certificate Chain

A certificate rarely stands alone. It is typically presented alongside one or more **intermediate certificates**, forming a chain back to a trusted root.

See the PKI guide's Section 4 for the full trust-chain mechanics.

Certificate Chain

```
End-Entity Certificate
(e.g., www.example.com)
    |
    | signed by
    ▼
Intermediate CA Certificate
    |
    | signed by
    ▼
Root CA Certificate
(self-signed, pre-trusted)
```

A relying party, such as a browser, verifies each signature in the chain, ultimately checking that the root is present in its own **trusted certificate store**.

6. Certificate Formats and Encodings

Format	Description
--------	-------------

PEM	Base64-encoded, text-based format (<code>-----BEGIN CERTIFICATE-----</code>), the most common format for web servers.
DER	Binary-encoded equivalent of PEM.
PKCS#12 / PFX	A binary format bundling a certificate together with its private key, typically password-protected — used for import/export between systems.
CSR (Certificate Signing Request)	Not a certificate itself , but the request document a subscriber submits to a CA, containing their public key and identifying information, ahead of signing.

7. Self-Signed Certificates

A **self-signed certificate** is signed by its own private key rather than by a separate CA.

This means it carries **no third-party vouching** for its identity claim.

Common Uses

Self-signed certificates are common in:

- Development and testing environments
- Internal systems within an organization that has manually distributed trust for that specific certificate

Limitation

Self-signed certificates are **not suitable for public-facing production use**, since a relying party has no independent basis to trust the identity claim without a recognized CA's signature.

8. Certificate Pinning

Certificate pinning (or public-key pinning) is a technique where a client hard-codes or pre-configures the expected certificate or public key for a specific service.

The client rejects connections presenting any other certificate — even one that would otherwise validate successfully through the normal CA chain.

What Pinning Defends Against

Pinning can defend against a scenario where an attacker has:

- Compromised a Certificate Authority, or
- Coerced a CA into issuing a fraudulent but chain-valid certificate for the target domain.

Trade-Off

Certificate pinning comes at the cost of **operational fragility**.

A pinned client can break if the legitimate certificate is rotated without also updating the pin.

Key point: Pinning provides an additional trust restriction beyond the normal CA validation chain, but requires careful operational management.

9. Typical Use Cases

Digital certificates are commonly used for:

- **TLS/HTTPS server authentication** and, less commonly, client authentication.
- **Code-signing certificates** for software publishers.
- **S/MIME certificates** for email encryption and signing.
- **Client certificates for mutual TLS (mTLS)** authentication in enterprise and machine-to-machine contexts.
- **Document and PDF signing certificates** for legal/compliance purposes.

10. Summary

A digital certificate is the **concrete, signed document that makes PKI's abstract trust model operational**.

It:

- Binds a **public key to a verified identity**.
- Uses a **Certificate Authority's digital signature** to establish that binding.
- Is structured according to the **X.509 standard**.
- Can be chained back to a **trusted root**.

What a Certificate Attests To

A certificate's trust should be understood in terms of:

1. **Identity/domain control**
2. **The time of issuance**
3. **The specific validation depth used**

What a Certificate Does Not Prove

A valid certificate does **not** automatically mean that:

- The website is trustworthy.
- The website is safe.
- The website contains no malicious content.

Bottom line: A certificate establishes a trusted binding between a public key and an identity; it does not, by itself, establish that the identified entity is trustworthy or safe.